

Nombre: *Jair Patiño*



In [1]:
```
import numpy as np
from sklearn import datasets
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import KFold
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

In [2]:
```
# Carga de datos.
iris = datasets.load_iris()
print(iris.target_names)
```

```
['setosa' 'versicolor' 'virginica']
```

In [3]:
```
# Mostrar características de la tabla de datos.
print("Tabla de datos: %d instancias y %d atributos" % (iris.data.shape[0], iris.da
print("Valores de la clase:", set(iris.target))
```

```
Tabla de datos: 150 instancias y 4 atributos
Valores de la clase: {np.int64(0), np.int64(1), np.int64(2)}
```

In [4]:
```
# Test: hold-out split 80-20%. PARTICIÓN EXTERNA
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)
print(X_train.shape, X_test.shape)
```

```
(120, 4) (30, 4)
```

In [5]:
```
# Estandarizar las características de entrenamiento y de test
standardizer = StandardScaler()
X_train = standardizer.fit_transform(X_train)
X_test = standardizer.transform(X_test)
```

In [6]:
```
# Definimos el algoritmo SVM para clasificación
from sklearn.svm import SVC
svc = SVC(C=0.5, gamma='auto')

# Hacemos el cross-validation interno para seleccionar los mejores hiperparámetros
results = cross_val_score(svc, X_train, y_train, cv = KFold(n_splits=5))
print("Resultados por bolsa: ", results)
print("Accuracy (media +/- desv.): %0.4f +/- %0.4f" % (results.mean(), results.std(
```

```
Resultados por bolsa:  [0.95833333 0.95833333 0.83333333 0.95833333 0.95833333]
Accuracy (media +/- desv.): 0.9333 +/- 0.0500
```

In [7]:
```
# Una vez entrenado y validado el modelo para seleccionar los mejores hyperparamete
# "train" y "val" para entrenar el modelo definitivo
```

```
svc = SVC(C=0.5, gamma='auto')
svc.fit(X_train, y_train)
```

Out[7]:

SVC

▶ Parameters

In [8]:
```
# Calcular la accuracy del conjunto de test
test_results = svc.score(X_test, y_test)
print('Exactitud en test: ', test_results*100, '%')
```

```
Exactitud en test:  100.0 %
```

In [9]:
```
# Extraer las predicciones, en lugar de directamente la accuracy
y_pred = svc.predict(X_test)
print('Predicciones:     ', y_pred)
print('Etiquetas reales: ', y_test)
```

```
Predicciones:      [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Etiquetas reales:  [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

GITHUB

- https://github.com/Failedbanjo/machine_learning_2

In [ ]:
```
```